The database transaction code you added in *Isolating Your Specs Using Database Transactions*, on page 92 is an example of the first situation:

07-structuring-code-examples/05/expense_tracker/spec/support/db.rb

```
RSpec.configure do |c|
  # ...

  c.around(:example, :db) do |example|
    DB.transaction(rollback: :always) { example.run }
  end
end
```

By putting your transaction rollback in an `around` hook, you avoided littering each database-dependent spec with this transaction logic.

In *Handling Failure*, on page 73, you wrote the second kind of hook:

07-structuring-code-examples/05/expense_tracker/spec/unit/app/api_spec.rb

```
context 'when the expense fails validation' do
  # ...

  before do
    allow(ledger).to receive(:record)
      .with(expense)
      .and_return(RecordResult.new(false, 417, 'Expense incomplete'))
  end

  # ...
end
```

Your `before` hook translates the description `'when the expense fails validation'` into Ruby code. Inside your hook, you configure the `ledger` test double to *fail* to record the expense. In doing so, you ensure that the description holds true for all the examples inside the context. You also make it easy for a reader to see what the English description means in terms of your domain model.

A hook should make it easier to follow your examples. Abusing RSpec hooks will make you skip all over your `spec` directory to trace the program flow.

In the next section, we'll show you how to recognize a bad hook and deal with it using another organizational tool: helper methods.

Helper Methods

Sometimes, we can get too clever for our own good and misuse these constructs in an effort to remove every last bit of repetition from our specs. Have a look at the following spec. It's quite complex, so we'll go through it in detail afterward.

07-structuring-code-examples/06/transit/spec/berlin_transit_ticket_spec.rb

```
Line 1 RSpec.describe BerlinTransitTicket do
```